1. Data Structures in Python

A **data structure** is a way of storing and organizing data so that it can be used efficiently.

In Python, the main built-in data structures are:

- List
- Tuple
- Set
- Dictionary

You can think of them like different types of containers.

1.1 List

A **list** stores multiple items in a single variable.

Features of List

- Ordered
- Changeable (mutable)
- Allows duplicate values
- Uses square brackets `[ ]`

Example

```
students = ["Arun", "Bala", "Charan"]
print(students)
```

Output

```
['Arun', 'Bala', 'Charan']
```

Accessing list items

```
students = ["Arun", "Bala", "Charan"]
print(students[0])   # first item
print(students[1])   # second item
print(students[-1])  # last item
```

Changing list values

```
students = ["Arun", "Bala", "Charan"]
students[1] = "Bharath"
print(students)
```

Adding items

```
students = ["Arun", "Bala"]
students.append("Charan")
print(students)
```

Inserting at specific position

```
students = ["Arun", "Bala"]
students.insert(1, "David")
print(students)
```

Removing items

```
students = ["Arun", "Bala", "Charan"]
students.remove("Bala")
print(students)
```

Removing by index

```
students = ["Arun", "Bala", "Charan"]
students.pop(1)
print(students)
```

Looping through a list

```
students = ["Arun", "Bala", "Charan"]

for name in students:
  print(name)
```

Useful list methods

```
numbers = [10, 20, 30, 40]

print(len(numbers))     # length
print(max(numbers))     # maximum
print(min(numbers))     # minimum
print(sum(numbers))     # total
```

1.2 Tuple

A **tuple** is similar to a list, but it cannot be changed after creation.

Features of Tuple

- Ordered
- Not changeable (immutable)
- Allows duplicate values
- Uses parentheses ( )

Example

```
colors = ("red", "green", "blue")
print(colors)
```

Accessing tuple items

```
colors = ("red", "green", "blue")
print(colors[0])
print(colors[-1])
```

Why use tuple?

Use tuple when the values should not change.

Examples:

- Days of week
- Months of year
- Fixed coordinates

Example

```
days = ("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")
print(days)
```

Tuple with one item

This is important:

```
x = ("apple",)
print(type(x))
```

Without comma, Python will treat it like a normal string inside brackets:

```
x = ("apple")
print(type(x))   # str
```

1.3 Set

A **set** is a collection of unique values.

Features of Set

- Unordered
- No duplicate values
- Changeable
- Uses curly braces `{}`

Example

```
numbers = {10, 20, 30, 20, 10}
print(numbers)
```

Output

```
{10, 20, 30}
```

Duplicates are automatically removed.

Adding item to set

```
numbers = {10, 20, 30}
numbers.add(40)
print(numbers)
```

Removing item

```
numbers = {10, 20, 30}
numbers.remove(20)
print(numbers)
```

Use of set

Set is useful when:

- You want unique values only
- You want to remove duplicates from a list
- You want mathematical operations like union and intersection

Example: remove duplicates from list

```
marks = [90, 80, 90, 70, 80]
unique_marks = set(marks)
print(unique_marks)
```

Set operations

```
a = {1, 2, 3}
b = {3, 4, 5}

print(a.union(b))         # combine
print(a.intersection(b))  # common values
print(a.difference(b))    # values in a but not in b
```

1.4 Dictionary

A **dictionary** stores data in **key-value pairs**.

Features of Dictionary

- Ordered
- Changeable
- No duplicate keys
- Uses curly braces `{}`

Example

```
student = {
  "id": 101,
  "name": "Arun",
  "course": "Python"
}

print(student)
```

Accessing dictionary values

```
student = {
  "id": 101,
  "name": "Arun",
  "course": "Python"
}

print(student["name"])
print(student["course"])
```

Changing values

```
student = {
  "id": 101,
```

```
  "name": "Arun",
  "course": "Python"
}

student["course"] = "Data Science"
print(student)
```

Adding new key-value pair

```
student = {
  "id": 101,
  "name": "Arun"
}

student["city"] = "Chennai"
print(student)
```

Removing item

```
student = {
  "id": 101,
  "name": "Arun",
  "city": "Chennai"
}

student.pop("city")
print(student)
```

Looping through dictionary

```
student = {
  "id": 101,
  "name": "Arun",
  "course": "Python"
}

for key in student:
  print(key, ":", student[key])
```

Important methods

```
student = {
  "id": 101,
  "name": "Arun",
  "course": "Python"
}
```

```
print(student.keys())
print(student.values())
print(student.items())
```

1.5 Comparison of Data Structures

List

Use when:

- Order matters
- Data can change
- Duplicates are allowed

Tuple

Use when:

- Order matters
- Data should not change

Set

Use when:

- Only unique values are needed
- Order is not important

Dictionary

Use when:

- Data should be stored as key-value pairs

1.6 Real-world examples

List

```
cart_items = ["Laptop", "Mouse", "Keyboard"]
```

Tuple

location = (12.9716, 77.5946)

Set

unique_courses = {"Python", "Java", "Python", "C#"}

Dictionary

```
employee = {
  "emp_id": 1001,
  "emp_name": "Ganesh",
  "department": "Training"
}
```

---

2. String Handling in Python

A **string** is a sequence of characters.

Examples:

- `"Hello"`
- `'Python'`
- `"12345"`
- `"Welcome to Python"`

Strings are one of the most commonly used data types.

2.1 Creating strings

```
name = "Python"
city = 'Chennai'
message = """Welcome to
Python programming"""
```

2.2 Accessing characters

Each character has an index position.

word = "Python"

```
print(word[0])   # P
print(word[1])   # y
print(word[-1])  # n
```

2.3 String slicing

Slicing means extracting part of a string.

```
word = "Python"

print(word[0:2])   # Py
print(word[2:6])   # thon
print(word[:4])    # Pyth
print(word[3:])    # hon
```

2.4 String is immutable

Strings cannot be changed directly.

```
name = "python"
# name[0] = "P"   # error
```

Correct way:

```
name = "python"
name = "P" + name[1:]
print(name)
```

2.5 String methods

`upper()`

```
text = "hello"
print(text.upper())
```

`lower()`

```
text = "HELLO"
print(text.lower())
```

title()

```
text = "welcome to python"
print(text.title())
```

strip()

Removes spaces from beginning and end.

```
text = "   hello   "
print(text.strip())
```

replace()

```
text = "I like Java"
print(text.replace("Java", "Python"))
```

split()

Converts string into list.

```
text = "apple,banana,mango"
print(text.split(","))
```

join()

Joins list items into one string.

```
words = ["Python", "is", "easy"]
result = " ".join(words)
print(result)
```

find()

Returns index of first occurrence.

```
text = "welcome to python"
print(text.find("python"))
```

count()

Counts occurrences.

```
text = "python python java python"
print(text.count("python"))
```

`startswith()` and `endswith()`

```
filename = "report.pdf"
print(filename.startswith("rep"))
print(filename.endswith(".pdf"))
```

2.6 String concatenation

Joining strings together.

```
first_name = "Arun"
last_name = "Kumar"
full_name = first_name + " " + last_name
print(full_name)
```

2.7 String repetition

```
print("Hi " * 3)
```

2.8 String formatting

String formatting means inserting values into strings.

Method 1: concatenation

```
name = "Arun"
age = 25
print("My name is " + name + " and I am " + str(age) + " years old")
```

Method 2: `format()`

```
name = "Arun"
age = 25
print("My name is {} and I am {} years old".format(name, age))
```

Method 3: f-string

Best and most modern approach.

```
name = "Arun"
age = 25
print(f"My name is {name} and I am {age} years old")
```

2.9 Checking string contents

`isalpha()`

```
print("Python".isalpha())   # True
```

`isdigit()`

```
print("12345".isdigit())    # True
```

`isalnum()`

```
print("Python123".isalnum())  # True
```

`isspace()`

```
print("   ".isspace())  # True
```

2.10 Loop through string

```
text = "Python"

for ch in text:
  print(ch)
```

2.11 Real-world examples

Example: count vowels

```
text = input("Enter a string: ")
count = 0

for ch in text.lower():
  if ch in "aeiou":
      count += 1

print("Number of vowels:", count)
```

Example: reverse a string

```
text = input("Enter a string: ")
```

```
print("Reversed string:", text[::-1])
```

Example: palindrome check

A palindrome reads same forward and backward.

```
text = input("Enter a word: ")

if text == text[::-1]:
  print("Palindrome")
else:
  print("Not a palindrome")
```

---

3. Modules and Packages

As programs grow, writing everything in one file becomes difficult. Python solves this by using:

- Modules
- Packages

These help organize code better.

3.1 What is a Module?

A **module** is a Python file containing functions, variables, or classes.

For example, if you create a file called `math_utils.py`, that file itself is a module.

Example: `math_utils.py`

```
def add(a, b):
  return a + b

def sub(a, b):
  return a - b
```

Now use it in another file.

Example: `main.py`

```
import math_utils
```

```
print(math_utils.add(10, 5))
print(math_utils.sub(10, 5))
```

3.2 Importing module

Import full module

```
import math
print(math.sqrt(25))
```

Import specific function

```
from math import sqrt
print(sqrt(36))
```

Import with alias

```
import math as m
print(m.pi)
```

Import multiple items

```
from math import sqrt, pi
print(sqrt(49))
print(pi)
```

3.3 Built-in modules

Python provides many ready-made modules.

`math`

```
import math

print(math.sqrt(16))
print(math.pow(2, 3))
print(math.factorial(5))
```

`random`

```
import random

print(random.randint(1, 10))
```

```
print(random.choice(["red", "green", "blue"]))
```

datetime

```
import datetime

today = datetime.datetime.now()
print(today)
```

os

```
import os

print(os.getcwd())
```

3.4 What is a Package?

A **package** is a collection of modules stored in a folder.

A package helps organize related modules together.

Example structure

```
mypackage/
  __init__.py
  calc.py
  message.py
```

- `mypackage` is the package
- `calc.py` and `message.py` are modules
- `__init__.py` tells Python this folder can be treated as a package

3.5 Example of package usage

calc.py

```
def add(a, b):
  return a + b
```

message.py

```
def greet(name):
```

```
    return f"Hello, {name}"
```

`main.py`

```
from mypackage.calc import add
from mypackage.message import greet

print(add(10, 20))
print(greet("Arun"))
```

3.6 Why use modules and packages?

They help in:

- Code reuse
- Better readability
- Easier testing
- Easier maintenance
- Separation of logic

For example:

- one file for database code
- one file for business logic
- one file for input/output handling

That is much cleaner than putting everything in one file.

3.7 Real-world example

Suppose you build a student management system.

You may organize like this:

```
student_app/
  __init__.py
  student.py
  marks.py
  reports.py
  main.py
```

- `student.py` → student details

- `marks.py` → marks calculations
- `reports.py` → report generation
- `main.py` → run program

This is how real applications are structured.

---

4. File Handling in Python

File handling means working with files such as:

- text files
- CSV files
- log files
- configuration files

With file handling, we can:

- create files
- read data from files
- write data to files
- append data to files

4.1 Why file handling is needed

Without file handling, data is lost once program stops.

For example:

- student records
- employee details
- log messages
- reports
- saved settings

All these need files or databases.

4.2 Opening a file

Python uses `open()` function.

Syntax

open("filename", "mode")

Common modes

- `"r"` → read
- `"w"` → write
- `"a"` → append
- `"x"` → create
- `"rb"` → read binary
- `"wb"` → write binary

4.3 Reading a file

Suppose `data.txt` contains:

Hello
Welcome to Python

Example

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

Important

Always close the file after use.

4.4 Writing to a file

```
file = open("data.txt", "w")
file.write("Hello\n")
file.write("Welcome to Python")
file.close()
```

If file does not exist, it will be created.
If it exists, old content will be overwritten.

4.5 Appending to a file

```
file = open("data.txt", "a")
file.write("\nThis is new line")
file.close()
```

Append adds content to the end without removing old content.

4.6 Using `with open()`

Best practice is to use `with` because it automatically closes the file.

Reading

```
with open("data.txt", "r") as file:
  content = file.read()
  print(content)
```

Writing

```
with open("data.txt", "w") as file:
  file.write("Python file handling example")
```

4.7 Reading line by line

```
with open("data.txt", "r") as file:
  for line in file:
    print(line.strip())
```

4.8 `read()`, `readline()`, `readlines()`

`read()`

Reads entire content.

```
with open("data.txt", "r") as file:
    print(file.read())
```

`readline()`

Reads one line at a time.

```
with open("data.txt", "r") as file:
    print(file.readline())
    print(file.readline())
```

`readlines()`

Returns all lines as a list.

```
with open("data.txt", "r") as file:
    lines = file.readlines()
    print(lines)
```

4.9 File exists check

Using `os` module.

```
import os

if os.path.exists("data.txt"):
    print("File exists")
else:
    print("File not found")
```

4.10 Real-world example: student record save

```
name = input("Enter student name: ")
marks = input("Enter marks: ")

with open("students.txt", "a") as file:
    file.write(f"{name},{marks}\n")

print("Record saved successfully.")
```

4.11 Real-world example: read student records

```
with open("students.txt", "r") as file:
  for line in file:
      data = line.strip().split(",")
      print("Name:", data[0], "Marks:", data[1])
```

4.12 Important notes

Difference between write and append

- `"w"` replaces old data
- `"a"` keeps old data and adds new data

File path

You can use:

```
with open("C:/Users/Admin/Documents/data.txt", "r") as file:
  print(file.read())
```

On Windows, forward slash works well in Python.

---

5. Exception Handling

An **exception** is an error that occurs while a program is running.

If exceptions are not handled, the program may crash.

Exception handling allows us to manage such errors gracefully.

5.1 Common examples of exceptions

Division by zero

```
print(10 / 0)
```

Invalid conversion

```
num = int("abc")
```

File not found

```
with open("abc.txt", "r") as file:
  print(file.read())
```

These cause runtime errors.

5.2 Why exception handling is needed

Without exception handling:

- program stops suddenly
- user gets confusing error message
- application becomes unreliable

With exception handling:

- program continues safely
- user gets proper message
- errors can be logged and controlled

5.3 `try` and `except`

Syntax

```
try:
  # risky code
except:
  # error handling code
```

Example

```
try:
  num = int(input("Enter a number: "))
  print(100 / num)
except:
  print("Invalid input or division by zero")
```

5.4 Handling specific exceptions

This is better than using plain `except`.

```
try:
  num = int(input("Enter a number: "))
  result = 100 / num
  print(result)
except ValueError:
  print("Please enter only numeric value")
except ZeroDivisionError:
  print("Number should not be zero")
```

5.5 `else` block

`else` runs if no exception occurs.

```
try:
  num = int(input("Enter a number: "))
  result = 100 / num
except ValueError:
  print("Invalid number")
except ZeroDivisionError:
  print("Cannot divide by zero")
else:
  print("Result is:", result)
```

5.6 `finally` block

`finally` always runs whether exception occurs or not.

```
try:
  file = open("data.txt", "r")
  print(file.read())
except FileNotFoundError:
  print("File not found")
finally:
  print("This block always executes")
```

5.7 Full example

```
try:
  a = int(input("Enter first number: "))
  b = int(input("Enter second number: "))
  result = a / b
except ValueError:
  print("Please enter valid integers only")
except ZeroDivisionError:
  print("Second number should not be zero")
else:
  print("Result:", result)
finally:
  print("Program ended")
```

5.8 Raising exceptions manually

We can create our own exception using `raise`.

```
age = int(input("Enter age: "))

if age < 18:
  raise ValueError("Age must be 18 or above")

print("Eligible")
```

This is useful when you want to enforce business rules.

5.9 Custom exception

You can define your own exception class.

```
class InvalidMarksError(Exception):
  pass

marks = int(input("Enter marks: "))

if marks < 0 or marks > 100:
  raise InvalidMarksError("Marks should be between 0 and 100")
```

5.10 Real-world example: ATM withdrawal

```
balance = 5000

try:
  amount = float(input("Enter withdrawal amount: "))

  if amount <= 0:
     raise ValueError("Amount must be greater than zero")

  if amount > balance:
     raise Exception("Insufficient balance")

  balance -= amount
  print("Withdrawal successful")
  print("Remaining balance:", balance)

except ValueError as ve:
  print("Value Error:", ve)

except Exception as e:
  print("Error:", e)
```

Combined Practical Understanding

Let us connect all five topics together.

Suppose you build a simple student record program:

- **Data Structures** → store student details in list/dictionary
- **String Handling** → format names, split lines, validate text
- **Modules and Packages** → separate code into files
- **File Handling** → save and read student records
- **Exception Handling** → handle invalid marks, missing file, wrong input

This is how Python topics work together in real programs.

Mini Integrated Example

```
def save_student():
    try:
        student_id = int(input("Enter student id: "))
        name = input("Enter student name: ").strip().title()
        marks = float(input("Enter marks: "))

        if marks < 0 or marks > 100:
            raise ValueError("Marks must be between 0 and 100")

        with open("students.txt", "a") as file:
            file.write(f"{student_id},{name},{marks}\n")

        print("Student record saved successfully.")

    except ValueError as e:
        print("Error:", e)

def display_students():
    try:
        with open("students.txt", "r") as file:
            for line in file:
                data = line.strip().split(",")
                print(f"ID: {data[0]}, Name: {data[1]}, Marks: {data[2]}")
    except FileNotFoundError:
        print("No student records found.")

save_student()
display_students()
```

Summary

Data Structures

Used to store collections of data.

- List → ordered, changeable
- Tuple → ordered, fixed
- Set → unique values
- Dictionary → key-value pairs

String Handling

Used to process text.

- slicing
- formatting
- searching
- replacing
- splitting
- joining

Modules and Packages

Used to organize large programs.

- module = one Python file
- package = folder of modules

File Handling

Used to store data permanently.

- read
- write
- append
- use `with open()`

Exception Handling

Used to prevent program crashes and manage runtime errors.

- `try`
- `except`
- `else`
- `finally`
- `raise`

Interview-style Questions

1. What is the difference between list and tuple?

List is mutable, tuple is immutable.

2. Why is dictionary useful?

Because it stores data in key-value form and allows fast access using keys.

3. What is the difference between `split()` and `join()`?

- `split()` converts string to list
- `join()` converts list to string

4. What is a module in Python?

A Python file containing reusable code.

5. Why use `with open()` in file handling?

Because it closes the file automatically.

6. What is the difference between syntax error and exception?

- Syntax error happens before execution
- Exception happens during execution

7. Why handle specific exceptions instead of generic except?

Because it gives better control and clearer error handling.

Suggested lab practice

Practice these programs one by one:

1. Create a list of 5 student names and print them
2. Store employee details in dictionary
3. Count vowels in a string
4. Reverse a string
5. Create your own module with add and multiply functions
6. Write text into a file and read it back

7. Create a student record file program
8. Write a division program with exception handling
9. Validate age using `raise`
10. Build a mini menu-driven student management program

Python Hands-on Lab

LAB 1: Working with Data Structures

Objective

Understand how to use:

- list
- tuple
- set
- dictionary

Exercise 1: List Basics

Task

Create a list of 5 student names and perform basic operations.

Code

```
students = ["Arun", "Bala", "Charan", "Divya", "Eswar"]

print("Original list:", students)

print("First student:", students[0])
print("Last student:", students[-1])

students.append("Farhan")
print("After append:", students)

students.insert(2, "Gokul")
print("After insert:", students)
```

```
students.remove("Bala")
print("After remove:", students)

print("Total students:", len(students))
```

Expected Learning

Learner understands:

- indexing
- append
- insert
- remove
- length

Exercise 2: List Looping

Task

Print each item in a list using loop.

Code

```
courses = ["Python", "Java", ".NET", "Angular"]

for course in courses:
  print("Course:", course)
```

Observation

This is useful when data is stored as multiple values and we want to process each item one by one.

Exercise 3: Tuple Basics

Task

Store days of week in a tuple.

Code

```
days = ("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")

print("Days:", days)
print("First day:", days[0])
print("Weekend day:", days[5])
```

Observation

Tuple is used when data should remain fixed.

Exercise 4: Set Basics

Task

Remove duplicate values from marks list.

Code

```
marks = [90, 85, 90, 70, 85, 100, 70]

unique_marks = set(marks)

print("Original list:", marks)
print("Unique marks:", unique_marks)
```

Observation

Set automatically removes duplicates.

Exercise 5: Dictionary Basics

Task

Store employee details in a dictionary.

Code

```
employee = {
  "id": 101,
```

```
  "name": "Ganesh",
  "department": "Training",
  "salary": 50000
}

print("Employee details:")
print("ID:", employee["id"])
print("Name:", employee["name"])
print("Department:", employee["department"])
print("Salary:", employee["salary"])
```

Exercise 6: Loop through Dictionary

Code

```
employee = {
  "id": 101,
  "name": "Ganesh",
  "department": "Training",
  "salary": 50000
}

for key, value in employee.items():
  print(key, ":", value)
```

Observation

Dictionary is very useful for structured data like:

- employee record
- student record
- product record
- configuration settings

LAB 2: String Handling

Objective

Understand string operations such as:

- indexing
- slicing
- formatting
- split/join
- replace
- validation

Exercise 1: Access characters from string

Code

```
text = "Python Programming"

print("Original text:", text)
print("First character:", text[0])
print("Last character:", text[-1])
print("First 6 characters:", text[0:6])
print("Programming word:", text[7:])
```

Exercise 2: Convert string cases

Code

```
message = "welcome to python"

print("Original:", message)
print("Uppercase:", message.upper())
print("Lowercase:", message.lower())
print("Title case:", message.title())
```

Exercise 3: Remove extra spaces

Code

```
name = "   ganesh   "

print("Before strip:", name)
print("After strip:", name.strip())
```

Observation

Useful for user input validation.

Exercise 4: Replace text

Code

```
sentence = "I am learning Java"

new_sentence = sentence.replace("Java", "Python")

print("Old sentence:", sentence)
print("New sentence:", new_sentence)
```

Exercise 5: Split and Join

Code

```
data = "apple,banana,mango,grapes"
items = data.split(",")

print("Items list:", items)

joined_text = " | ".join(items)
print("Joined text:", joined_text)
```

Observation

This is commonly used in:

- CSV processing
- file reading
- user input parsing

Exercise 6: Count vowels in a string

Code

```
text = input("Enter a string: ")
count = 0
```

```
for ch in text.lower():
  if ch in "aeiou":
      count += 1

print("Number of vowels:", count)
```

Exercise 7: Reverse a string

Code

```
text = input("Enter a word: ")
print("Reversed:", text[::-1])
```

Exercise 8: Palindrome check

Code

```
text = input("Enter a word: ").lower()

if text == text[::-1]:
  print("It is a palindrome")
else:
  print("It is not a palindrome")
```

Examples:

- madam
- level
- radar

Exercise 9: String formatting

Code

```
name = "Arun"
age = 24
course = "Python"

print(f"My name is {name}, I am {age} years old, and I am learning {course}.")
```

Observation

f-string is the cleanest and most modern way.

LAB 3: Modules and Packages

Objective

Learn how to organize code into separate files.

Exercise 1: Create your first module

Step 1: Create a file named `calculator.py`

`calculator.py`

```
def add(a, b):
  return a + b

def subtract(a, b):
  return a - b

def multiply(a, b):
  return a * b

def divide(a, b):
  return a / b
```

Step 2: Create another file named `main.py`

`main.py`

```
import calculator

print("Addition:", calculator.add(10, 5))
print("Subtraction:", calculator.subtract(10, 5))
print("Multiplication:", calculator.multiply(10, 5))
print("Division:", calculator.divide(10, 5))
```

Observation

This is a module because `calculator.py` contains reusable functions.

Exercise 2: Import specific functions

`main.py`

```
from calculator import add, multiply

print("Addition:", add(20, 10))
print("Multiplication:", multiply(4, 5))
```

Exercise 3: Use built-in modules

Math module

```
import math

print("Square root of 25:", math.sqrt(25))
print("Power:", math.pow(2, 3))
print("Pi value:", math.pi)
```

Random module

```
import random

print("Random number between 1 and 10:", random.randint(1, 10))
print("Random course:", random.choice(["Python", "Java", ".NET"]))
```

Datetime module

```
import datetime

now = datetime.datetime.now()
print("Current date and time:", now)
```

Exercise 4: Create a package

Folder structure

```
mypackage/
  __init__.py
  message.py
  maths.py
main.py
```

`message.py`

```
def greet(name):
  return f"Hello, {name}"
```

`maths.py`

```
def square(n):
  return n * n
```

`main.py`

```
from mypackage.message import greet
from mypackage.maths import square

print(greet("Ganesh"))
print("Square of 6:", square(6))
```

Observation

Package helps organize related modules in a folder.

LAB 4: File Handling

Objective

Learn to:

- create file
- write file
- read file
- append file
- process data from file

Exercise 1: Write into a file

Code

```
with open("sample.txt", "w") as file:
  file.write("Welcome to Python File Handling\n")
  file.write("This is the second line\n")

print("Data written successfully.")
```

Observation

If file does not exist, Python creates it.
If file exists, old data is overwritten.

Exercise 2: Read a file

Code

```
with open("sample.txt", "r") as file:
  content = file.read()

print("File content:")
print(content)
```

Exercise 3: Read line by line

Code

```
with open("sample.txt", "r") as file:
  for line in file:
    print(line.strip())
```

Exercise 4: Append data to file

Code

```
with open("sample.txt", "a") as file:
  file.write("This line is appended later.\n")
```

```
print("Data appended successfully.")
```

Exercise 5: Store student records in file

Code

```
name = input("Enter student name: ")
marks = input("Enter student marks: ")

with open("students.txt", "a") as file:
  file.write(f"{name},{marks}\n")

print("Student record saved.")
```

Exercise 6: Read student records

Code

```
with open("students.txt", "r") as file:
  for line in file:
      data = line.strip().split(",")
      print("Name:", data[0], "| Marks:", data[1])
```

Observation

This is a simple file-based mini database approach.

Exercise 7: Count number of lines in file

Code

```
count = 0

with open("students.txt", "r") as file:
  for line in file:
      count += 1

print("Total records:", count)
```

Exercise 8: Search student in file

Code

```
search_name = input("Enter student name to search: ").lower()
found = False

with open("students.txt", "r") as file:
    for line in file:
        data = line.strip().split(",")
        if data[0].lower() == search_name:
            print("Record found:", line.strip())
            found = True
            break

if not found:
    print("Student not found.")
```

LAB 5: Exception Handling

Objective

Learn how to handle runtime errors safely.

Exercise 1: Division with exception handling

Code

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
    print("Result:", result)
except ValueError:
    print("Please enter valid integers only.")
except ZeroDivisionError:
    print("Cannot divide by zero.")
```

Observation

Program does not crash. It shows user-friendly message.

Exercise 2: File not found handling

Code

```
try:
  with open("unknown.txt", "r") as file:
      print(file.read())
except FileNotFoundError:
  print("The file does not exist.")
```

Exercise 3: Use else and finally

Code

```
try:
  num = int(input("Enter a number: "))
  result = 100 / num
except ValueError:
  print("Invalid input.")
except ZeroDivisionError:
  print("Cannot divide by zero.")
else:
  print("Successful result:", result)
finally:
  print("Program execution completed.")
```

Observation

- `else` runs only if no error
- `finally` always runs

Exercise 4: Raise custom validation error

Code

```
marks = int(input("Enter marks: "))
```

```
try:
  if marks < 0 or marks > 100:
      raise ValueError("Marks should be between 0 and 100")

  print("Valid marks entered.")
except ValueError as e:
  print("Error:", e)
```

Exercise 5: ATM withdrawal example

Code

```
balance = 5000

try:
  amount = float(input("Enter withdrawal amount: "))

  if amount <= 0:
      raise ValueError("Withdrawal amount must be greater than zero.")

  if amount > balance:
      raise Exception("Insufficient balance.")

  balance -= amount
  print("Withdrawal successful.")
  print("Remaining balance:", balance)

except ValueError as ve:
  print("Value Error:", ve)
except Exception as e:
  print("Error:", e)
```

Mini Project Lab: Student Record Management

Now let us combine all five topics into one simple mini project.

Features

- Add student record
- View all records
- Search student
- Handle invalid input
- Save data to file

Code

```
def add_student():
    try:
        student_id = int(input("Enter student ID: "))
        name = input("Enter student name: ").strip().title()
        marks = float(input("Enter student marks: "))

        if marks < 0 or marks > 100:
            raise ValueError("Marks must be between 0 and 100.")

        with open("student_records.txt", "a") as file:
            file.write(f"{student_id},{name},{marks}\n")

        print("Student record added successfully.\n")

    except ValueError as e:
        print("Error:", e, "\n")


def view_students():
    try:
        with open("student_records.txt", "r") as file:
            print("\n--- Student Records ---")
            for line in file:
                data = line.strip().split(",")
                print(f"ID: {data[0]}, Name: {data[1]}, Marks: {data[2]}")
            print()
    except FileNotFoundError:
        print("No student records found.\n")


def search_student():
    try:
        search_name = input("Enter student name to search: ").strip().lower()
        found = False
```

```
        with open("student_records.txt", "r") as file:
            for line in file:
                data = line.strip().split(",")
                if data[1].lower() == search_name:
                    print(f"Record Found -> ID: {data[0]}, Name: {data[1]}, Marks: {data[2]}\n")
                    found = True
                    break

        if not found:
            print("Student not found.\n")

    except FileNotFoundError:
        print("No student records found.\n")


def menu():
    while True:
        print("===== Student Record Management =====")
        print("1. Add Student")
        print("2. View Students")
        print("3. Search Student")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            add_student()
        elif choice == "2":
            view_students()
        elif choice == "3":
            search_student()
        elif choice == "4":
            print("Exiting program.")
            break
        else:
            print("Invalid choice. Please try again.\n")


menu()
```

What concepts are used in this mini project

Data Structures

Used indirectly while processing data line by line and storing split values.

String Handling

Used in:

- `strip()`
- `title()`
- `lower()`
- `split(",")`

Modules and Packages

You can later separate this into:

- `student_operations.py`
- `file_utils.py`
- `main.py`

File Handling

Used to store records permanently in `student_records.txt`

Exception Handling

Used to handle:

- invalid numeric input
- missing file
- invalid marks

Practice Assignments for Learners

Here are good assignments to give learners after this lab.

Assignment 1

Create a program to store 5 product names in a list and print them in uppercase.

Assignment 2

Accept a sentence from user and count:

- number of vowels
- number of consonants
- number of spaces

Assignment 3

Create your own module with functions:

- add
- subtract
- multiply
- divide

Then import and use it in another file.

Assignment 4

Create a file called `employees.txt` and store employee name and salary.

Assignment 5

Create a program that reads file content and handles file-not-found exception.

Assignment 6

Build a menu-driven employee record management program similar to the student example.

Common mistakes beginners make

Data Structures

- using wrong brackets
  - list `[ ]`
  - tuple `( )`
  - dictionary `{ }`

String Handling

- forgetting strings are immutable
- confusing `split()` and `join()`

Modules

- module file name mismatch
- saving files in different folder and expecting import to work automatically

File Handling

- using `"w"` instead of `"a"` and losing previous data
- forgetting file path issues

Exception Handling

- using plain `except:` for everything
- not handling specific exceptions properly

Interview Questions from These Topics

Data Structures

What is the difference between list, tuple, set, and dictionary?

String Handling

Why are strings immutable in Python?

Modules

What is the difference between a module and a package?

File Handling

What is the difference between `read()`, `readline()`, and `readlines()`?

Exception Handling

What is the difference between `try`, `except`, `else`, and `finally`?

Validation

Why is `raise` useful in Python applications?